Libraries and Extensions to CUDA

CSCD439/539 GPU Programming

Outline

- OpenACC(chapter 15)
- Thrust(chapter 16)

OpenACC

- The OpenACC Application Programming Interface(API) provides,
  - a set of compiler directives, library routines, and environment variables
  - that can be used to write data-parallel FORTRAN, C, and C++ programs,
  - that run on accelerator devices, including GPUs.

OpenACC

- The Idea of OpenACC is similar to OpenMP.
  - Programmers provides directives to compiler,
    - About which part of the program to be parallelized
  - Compiler then generate parallel code
  - Run time system will execute the parallel code.
  - These directives provided as comment-like hints.

OpenACC

```
void computeAcc(float *P, const float *M, const float *N, int Mh,
  int Mw, int Nw)
{

 #pragma acc parallel loop copyin(M[0:Mh*Mw]) copyin(N[0:Nw*Mw])
   copyout(P[0:Mh*Nw])
 for (int i=0; i<Mh; i++) {
     #pragma acc loop
     for (int j=0; j<Nw; j++) {
         float sum = 0;
         for (int k=0; k<Mw; k++) {
             float a = M[i*Mw+k];
             float b = N[k*Nw+j];
             sum += a*b;
         }
         P[i*Nw+j] = sum;
     }
 }
}
```

FIGURE 15.1

Our first OpenACC program.

OpenACC

- The code in the figure is almost identical to the sequential version, except for the two lines with # pragma.
- Advantage:
  - Abstracting away many parallel programming details for programmers.
    - Automatically generate parallel code
  - Code is more portable

OpenACC

- OpenACC programmers can often start with writing a sequential version and then annotate their sequential program with OpenACC directives.
- OpenACC provides an incremental path for moving legacy applications to accelerators.

OpenACC

- A non-OpenACC compileris not required to understand and process OpenACC directives,
  - therefore it can just ignore the directives and compile the rest of the program as usual.
- While it is possible to write OpenACC programs that give the same execution result as when the directives are ignored, this property does not hold automatically for all OpenACC programs.

Thrust(Chapter 16)

- **Thrust is a C++ template library for CUDA based on the Standard Template Library (STL).**
- **Thrust allows you to implement high performance parallel applications with minimal programming effort through a high-level interface**
  - **fully interoperable with CUDA C.**

Thrust

- Thrust provides a rich collection of data **parallel primitives** such as scan, sort, and reduce,
  - which can be composed together to implement complex algorithms with concise, readable source code.

Thrust

- By describing your computation in terms of these high-level abstractions,
  - you provide Thrust with the freedom to select the most efficient implementation automatically.
  - As a result, Thrust can be utilized in rapid prototyping of CUDA applications,
  - high productivity with robustness and absolutely high performance.

Thrust

- Thrust is included in this version of the CUDA Toolkit.
- Demo of vector1.cu and thrustFill.cu

Inter-Operate with Cuda

```
size_t N = 10;
// raw pointer to device memory
int * raw_ptr;
 cudaMalloc((void **) &raw_ptr, N * sizeof(int));
// wrap raw pointer with a device_ptr
thrust::device_ptr<int> dev_ptr(raw_ptr);
 // use device_ptr in thrust algorithms
thrust::fill(dev_ptr, dev_ptr + N, (int) 0);
```

Inter-Operate with Cuda

- size_t N = 10;
- // create a device_ptr
- thrust::device_ptr<int> dev_ptr = thrust::device_malloc<int>(N);
- // extract raw pointer from device_ptr int * raw_ptr = thrust::raw_pointer_cast(dev_ptr);

Sort with Thrust 1

```
#include <thrust/sort.h>
...
const int N = 6;
 int A[N] = {1, 4, 2, 8, 5, 7};
 thrust::sort(A, A + N);

// A is now {1, 2, 4, 5, 7, 8}
```

Sort with Thrust 2

```
#include <thrust/sort.h>
...
const int N = 6;
int    keys[N] = {  1,   4,   2,   8,   5,   7};
char values[N] = {'a', 'b', 'c', 'd', 'e', 'f'};

thrust::sort_by_key(keys, keys + N, values);

// keys is now   {  1,   2,   4,   5,   7,   8}
// values is now {'a', 'c', 'b', 'e', 'f', 'd'}
```

Sort with Thrust 3

```
#include <thrust/sort.h>
#include <thrust/functional.h>

...
const int N = 6;
int A[N] = {1, 4, 2, 8, 5, 7};

thrust::stable_sort(A, A + N, thrust::greater<int>());

// A is now {8, 7, 5, 4, 2, 1}
```

Reduction with Thrust

- The sum of an array is implemented with thrust::reduce as follows:

```
int sum = thrust::reduce(D.begin(), D.end(), (int) 0,
    thrust::plus<int>());
```

- The first two arguments to reduce define the range of values while the third and fourth parameters provide the initial value and reduction operator respectively.
- By default, last two parameters are optional. if no initial value or operator is provided.

Reduction with Thrust

- The following three lines are therefore equivalent:

```
int sum = thrust::reduce(D.begin(), D.end(), (int) 0,
                         thrust::plus<int>());

int sum = thrust::reduce(D.begin(), D.end(), (int) 0);
int sum = thrust::reduce(D.begin(), D.end())
```

Reduction with Thrust

- Thrust provides a few additional functions for convenience (like the STL).
  - Other reduction operations include thrust::count_if, thrust::min_element, thrust::max_element, thrust::is_sorted, thrust::inner_product, and several others.
- For example, thrust::count returns the number of instances of a specific value in a given sequence
  - Code snippet next slide

Reduction with Thrust

```
#include <thrust/count.h>
#include <thrust/device_vector.h>
...
// put three 1s in a device_vector
thrust::device_vector<int> vec(5,0);
vec[1] = 1;
vec[3] = 1;
vec[4] = 1;

// count the 1s
int result = thrust::count(vec.begin(), vec.end(), 1);
// result is three
```

Summary

- OpenACC
  - Programmer writes sequential code, then add directives,
  - Compiler uses directives to generate parallel code.
- Thrust
  - C++ library provides a lot of parallel primitives,
  - E.g Reduction, Sorting, high-level APIs